YOUR BUILD-ALONG WORKSHEET

Build Your AI Project Manager

Tick each checkpoint as it runs, jot the gotchas, then design your own crew.

Follow the live build from the code bundle. Use this sheet to confirm each step runs, capture what broke, and plan the agent crew you'll build after today.

A. Setup check (do before we build)

- ☐ `ollama run llama3.1` — replies with a word
- ☐ `pip install crewai crewai-tools pydantic` — done in an active venv
- ☐ Code bundle downloaded (pm_crew.py, virtual_office.py, brief.md)

B. Build checkpoints (tick as each runs)

- ☐ First agent runs and prints a plan
- ☐ save_tasks tool writes project_state.json
- ☐ Planner returns a typed Plan (Pydantic) — not prose
- ☐ Risk + Status-Writer agents added, context passed between tasks
- ☐ pm_crew.py runs end to end on brief.md
- ☐ virtual_office.py runs (hierarchical crew with a manager)

C. Notes & gotchas

What broke, and the fix (version mismatch, import, Ollama not running...):

Model + latency notes (which model, how slow, hardware):

D. Design your own crew (take home)

The project chore I'll automate first:

Agents (roles) I'd define — and the one job of each:

Tools each agent needs (calendar, repo, tracker, files, API):

Sequential or hierarchical? Why:

Where a human must stay in the loop:

E. Responsible-agent check

- ☐ Each agent has only the tools it truly needs (least privilege)
- ☐ Irreversible actions (send / pay / delete) require human approval
- ☐ No customer PII sent to third-party services (local-first)

Remember — the whole build in seven lines

1. An agent is an LLM plus tools, memory, and a loop (ReAct).
2. CrewAI runs that loop; Ollama runs the model locally.
3. Agent + Task + Crew is a working assistant in ~40 lines.
4. Tools are plain Python functions the agent may call.
5. Typed (Pydantic) outputs make agents composable.
6. A manager + specialists = a virtual office (hierarchical crew).
7. Guardrails, a human in the loop, and evals make it trustworthy.